

C++ Standard Library Priority 1 Issues Resolved Directly In Oulu

Doc. no. P0397R0

Date: 2016-06-24 Revised 2016-06-24 at 07:06:31 UTC

Project: Programming Language C++

Reply to: Alisdair Meredith <lwgchair@gmail.com>

Immediate Issues

[2687](#). {inclusive,exclusive}_scan misspecified

Section: 26.8.7 [exclusive.scan], 26.8.8 [inclusive.scan], 26.8.9 [transform.exclusive.scan], 26.8.10 [transform.inclusive.scan]

Status: [Immediate](#) **Submitter:** Tim Song **Opened:** 2016-03-21 **Last modified:** 2016-06-24

Priority: 1

View all issues with [Immediate](#) status.

Discussion:

When P0024R2 changed the description of {inclusive,exclusive}_scan (and the transform_ version), it seems to have introduced an off-by-one error. Consider what N4582 26.8.8 [inclusive.scan]/2 says of inclusive_scan:

Effects: Assigns through each iterator i in $[result, result + (last - first))$ the value of

- `GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *j, ...)` for every j in $[first, first + (i - result))$ if `init` is provided, or
- `GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, *j, ...)` for every j in $[first, first + (i - result))$ otherwise.

When $i == result$, $i - result = 0$, so the range $[first, first + (i - result))$ is $[first, first)$ — which is empty. Thus according to the specification, we should assign `GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init)` if `init` is provided, or `GENERALIZED_NONCOMMUTATIVE_SUM(binary_op)` otherwise, which doesn't seem "inclusive" — and isn't even defined in the second case.

The parallelism TS's version uses `GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, *first, ..., *(first + (i - result)))` — which is a closed range, not a half-open one.

Similar issues affect `exclusive_scan`, `transform_inclusive_scan`, and `transform_exclusive_scan`.

[2016-06 Oulu]

Voted to Ready 11-0 Tuesday evening in Oulu

Proposed resolution:

This wording is relative to N4582.

1. Edit 26.8.7 [exclusive.scan]/2 as indicated:

[Drafting note: when i is `result`, $[first, first + (i - result))$ is an empty range, so the value to be assigned reduces to `GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init)`, which is `init`, so there's no need to special case this.]

```
template<class InputIterator, class OutputIterator, class T, class BinaryOperation>
OutputIterator exclusive_scan(InputIterator first, InputIterator last,
                             OutputIterator result,
                             T init, BinaryOperation binary_op);
```

-2- Effects: Assigns through each iterator i in $[result, result + (last - first))$ the value of `GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *j, ...)` for every j in $[first, first + (i - result))$.

- ~~init, if i is result, otherwise~~
- ~~GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *j, ...) for every j in [first, first + (i - result) - 1];~~

2. Edit 26.8.8 [inclusive.scan]/2 as indicated:

```
template<class InputIterator, class OutputIterator, class BinaryOperation>
OutputIterator inclusive_scan(InputIterator first, InputIterator last,
                             OutputIterator result,
                             BinaryOperation binary_op);
template<class InputIterator, class OutputIterator, class BinaryOperation>
OutputIterator inclusive_scan(InputIterator first, InputIterator last,
                             OutputIterator result,
                             BinaryOperation binary_op, T init);
```

-2- *Effects*: Assigns through each iterator i in $[result, result + (last - first))$ the value of

- $GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, *j, \dots)$ for every j in $[first, first + (i - result + 1))$ if $init$ is provided, or
- $GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, *j, \dots)$ for every j in $[first, first + (i - result + 1))$ otherwise.

3. Edit 26.8.9 [transform.exclusive.scan]/1 as indicated:

```
template<class InputIterator, class OutputIterator,
         class UnaryOperation,
         class T, class BinaryOperation>
OutputIterator transform_exclusive_scan(InputIterator first, InputIterator last,
                                       OutputIterator result,
                                       UnaryOperation unary_op,
                                       T init, BinaryOperation binary_op);
```

-1- *Effects*: Assigns through each iterator i in $[result, result + (last - first))$ the value of $GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, unary_op(*j), \dots)$ for every j in $[first, first + (i - result))$.

- ~~init, if i is result, otherwise~~
- ~~GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, unary_op(*j), ...) for every j in [first, first + (i - result) - 1];~~

4. Edit 26.8.10 [transform.inclusive.scan]/1 as indicated:

```
template<class InputIterator, class OutputIterator,
         class UnaryOperation,
         class BinaryOperation>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                       OutputIterator result,
                                       UnaryOperation unary_op,
                                       BinaryOperation binary_op);
template<class InputIterator, class OutputIterator,
         class UnaryOperation,
         class BinaryOperation, class T>
OutputIterator transform_inclusive_scan(InputIterator first, InputIterator last,
                                       OutputIterator result,
                                       UnaryOperation unary_op,
                                       BinaryOperation binary_op, T init);
```

-1- *Effects*: Assigns through each iterator i in $[result, result + (last - first))$ the value of

- $GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, init, unary_op(*j), \dots)$ for every j in $[first, first + (i - result + 1))$ if $init$ is provided, or
- $GENERALIZED_NONCOMMUTATIVE_SUM(binary_op, unary_op(*j), \dots)$ for every j in $[first, first + (i - result + 1))$ otherwise.

2704. recursive_directory_iterator's members should require '*this is dereferenceable'

Section: 27.10.14.1 [rec.dir.itr.members], 27.10.13.1 [directory_iterator.members] **Status:** [Immediate](#) **Submitter:** Eric Fiselier
Opened: 2016-05-08 **Last modified:** 2016-06-24

Priority: 1

View other [active issues](#) in [rec.dir.itr.members].

View all other [issues](#) in [rec.dir.itr.members].

View all issues with [Immediate](#) status.

Discussion:

In 27.10.14.1 [rec.dir.itr.members] the following members are specified as having the requirement `"*this != recursive_directory_iterator{}"`:

- `options()`
- `depth()`
- `recursion_pending()`
- `operator++`
- `increment(...)`
- `pop()`
- `disable_recursion_pending()`

This requirement is not strong enough since it still allows non-dereferenceable iterators to invoke these methods. For example:

```
recursive_directory_iterator it(".");
recursive_directory_iterator it_copy(it);
assert(it_copy.depth() == 0); // OK
++it;
assert(it_copy.depth() == ???); // Not OK
auto x = *it_copy; // Is this OK?
```

I believe these should instead require that `*this` is dereferenceable, however the current specification seems to say that all previous copies of it are still dereferenceable although not what they dereference to.

[class.directory_iterator] p4:

The result of `operator*` on an end iterator is undefined behavior. For any other iterator value a `const recursive_directory_entry&` is returned. The result of `operator->` on an end iterator is undefined behavior. For any other iterator value a `const directory_entry*` is returned.

Is the intention of this clause to make all non-end iterators dereferenceable?

One further complication with these methods comes from the specification of `recursive_directory_iterator`'s copy/move constructors and assignment operators which specify the following post conditions:

- `this->options() == rhs.options()`
- `this->depth() == rhs.depth()`
- `this->recursion_pending() == rhs.recursion_pending()`

If `rhs` is the end iterator these post conditions are poorly stated.

[2016-06, Oulu — Daniel comments]

The loss of information caused by bullet three of the suggested wording below is corrected by [2726](#)'s wording.

Voted to Ready 7-0 Monday morning in Oulu

Proposed resolution:

This wording is relative to N4582.

[Drafting note: I have not attempted to fix the specification of the copy/move constructors and assignment

operators for recursive_directory_iterator]

[Drafting note: increment directly specifies "Effects: As specified by Input iterators (24.2.3)", so no additional specification is needed.]

1. Change 27.10.13 [class.directory_iterator] p4 as indicated:

~~-4- The result of operator* on an end iterator is undefined behavior. For any other iterator value a const directory_entry& is returned. The result of operator-> on an end iterator is undefined behavior. For any other iterator value a const directory_entry* is returned~~
The end iterator is not dereferenceable.

2. Add a new bullet after the class synopsis in 27.10.14 [class.rec.dir.itr]:

?- Calling options, depth, recursion_pending, pop OR disable_recursion_pending on an iterator that is not dereferencable results in undefined behavior.

3. Change 27.10.14.1 [rec.dir.itr.members] as indicated:

directory_options options() const;

~~-17- Requires: *this != recursive_directory_iterator();~~

[...]

int depth() const;

~~-20- Requires: *this != recursive_directory_iterator();~~

[...]

bool recursion_pending() const;

~~-23- Requires: *this != recursive_directory_iterator();~~

[...]

recursive_directory_iterator& operator++();

recursive_directory_iterator& increment(error_code& ec) noexcept;

~~-26- Requires: *this != recursive_directory_iterator();~~

[...]

void pop();

~~-30- Requires: *this != recursive_directory_iterator();~~

[...]

void disable_recursion_pending();

~~-32- Requires: *this != recursive_directory_iterator();~~

[...]

[2711](#). path is convertible from approximately everything under the sun

Section: 27.10.8.4.1 [path.construct] **Status:** [Immediate](#) **Submitter:** Tim Song **Opened:** 2016-05-10 **Last modified:** 2016-06-24

Priority: 1

View all issues with [Immediate](#) status.

Discussion:

The unconstrained template<class Source> path(const Source& source); constructor defines an implicit conversion from pretty much everything to path. This can lead to surprising ambiguities in overload resolution.

For example, given LWG [2676](#)'s proposed resolution, the following code appears to be ambiguous:

```
struct meow {
```

```
operator const char *() const;
};
```

```
std::ifstream purr(meow{});
```

because a `meow` can be converted to either a `path` or a `const char*` by a user-defined conversion, even though part of the stated goal of LWG 2676's P/R is to avoid "break[ing] user code depending on user-defined automatic conversions to the existing argument types".

Previous resolution [SUPERSEDED]:

This wording is relative to N4582.

[Drafting note: `decay_t<Source>` handles both the input iterator case (27.10.8.3 [path.req]/1.2) and the array case (27.10.8.3 [path.req]/1.3). The level of constraints required is debatable; the following wording limits the constraint to "is a `basic_string` or an iterator", but perhaps stronger constraints (e.g., an `iterator_category` check in the second case, and/or a `value_type` check for both cases) would be desirable.]

1. Change 27.10.8.4.1 [path.construct] as indicated:

```
template <class Source>
    path(const Source& source);
template <class InputIterator>
    path(InputIterator first, InputIterator last);
```

-4- *Effects*: Constructs an object of class `path`, storing the effective range of `source` (27.10.8.3) or the range `[first, last)` in `pathname`, converting format and encoding if required (27.10.8.2.1).

-?- *Remarks*: The first overload shall not participate in overload resolution unless either `Source` is a specialization of `basic_string` or the qualified-id `iterator_traits<decay_t<Source>>::value_type` is valid and denotes a type (14.8.2 [temp.deduct]).

[2016-05-28, Eric Fiselier comments suggests alternative wording]

Functions taking `EcharT` or `Source` parameter types often provide additional overloads with the same arity and concrete types. In order to allow conversions to these concrete types in the interface we need to explicitly disable the `EcharT` and `Source` overloads.

[2016-06-19, Eric and Tim improve the wording]

[2016-06, Oulu]

Voted to Ready 8-0 Monday morning in Oulu

Proposed resolution:

This wording is relative to N4594.

[Drafting note: Functions taking `EcharT` or `Source` parameter types often provide additional overloads with the same arity and concrete types. In order to allow conversions to these concrete types in the interface we need to explicitly disable the `EcharT` and `Source` overloads.]

1. Change 27.10.5 [fs.req] as indicated:

~~-2- Template parameters named `EcharT` shall be~~ Functions with template parameters named `EcharT` shall not participate in overload resolution unless `EcharT` is one of the encoded character types.

2. Insert a new paragraph between 27.10.8.3 [path.req] p1 and p2 as indicated:

-?- Functions taking template parameters named `Source` shall not participate in overload resolution unless either `Source` is a specialization of `basic_string` or the qualified-id `iterator_traits<decay_t<Source>>::value_type` is valid and denotes a possibly `const` encoded character type (14.8.2 [temp.deduct]).

[2725](#). filesystem::exists(const path&, error_code&) error reporting

Section: 27.10.15.12 [fs.op.exists] **Status:** [Immediate](#) **Submitter:** Jonathan Wakely **Opened:** 2016-06-06 **Last modified:** 2016-06-24

Priority: 1

View all issues with [Immediate](#) status.

Discussion:

The `filesystem::exists(const path&)` function does not throw an exception if the file doesn't exist, but the corresponding function taking an `error_code&` argument does set it to indicate an error.

It seems sensible for `filesystem::exists(const path&, error_code&)` to call `ec.clear()` if `status(p, ec).type() == file_type::not_found`.

[2016-06, Oulu — Jonathan comments and provides wording]

The sentence "The signature with argument `ec` returns false if an error occurs." means that given a file such that `status(p).type() == file_type::unknown`, `exists(p)` is true but `exists(p, ec)` is false.

I believe we should make the behaviour of `exists(p)` and `exists(p, ec)` consistent, so that the latter clears `ec` except when the former would throw an exception, which is only for the `file_type::none` case.

[2016-06, Oulu]

Prioritized as P1

Voted to Ready 7-0 Tuesday evening in Oulu

Proposed resolution:

This wording is relative to N4594.

1. Insert a new paragraph before 27.10.15.12 [fs.op.exists] p2 and edit it as shown:

```
bool exists(const path& p);  
bool exists(const path& p, error_code& ec) noexcept;
```

-?- Let `s` be a file status, determined as if by `status(p)` or `status(p, ec)`, respectively.

-?- Effects: The signature with argument `ec` calls `ec.clear()` if `status_known(s)`.

-2- Returns: `exists(status(p))` or `exists(status(p, ec))`, respectively `exists(s)`. The signature with argument `ec` returns false if an error occurs.